



BOARD GAME ASSIGNMENT

KALAH

AUTHORS

Group 44:

Emma Bindslev Schytte (s204246)

Thorri Halldorson (s252362)

Anna Noa Nørgaard (s201920)

Iben Jørgensen (s201974)

COURSE:

02180 Introduction to AI

DATE:

March 23, 2026

TABLE OF CONTENTS

1 Contributions	1
1.1 Group declaration	1
1.2 Percentage Contribution	1
2 Nomenclature	2
3 Programming Kalaha	3
3.1 Game rules	3
3.2 Formal Game Characterization	3
3.3 Implementation and State Representation	3
3.4 AI Algorithm: Minimax with Alpha-Beta Pruning	5
3.5 Performance results and Benchmarking	7
3.6 Discussion and Future Work	7
3.7 Conclusion	8
Bibliography	I
4 Appendix A	I
5 Appendix B	I

1 CONTRIBUTIONS

1.1 Group declaration

The work on this project was shared among all group members. A detailed description of the individual contributions is presented below.

- s201920:** I helped on how the code should be structured. I wrote the gameplay section, parts of the discussion and structured the report. Furthermore, I constructed the example tree visualizing alpha-beta pruning.
- s204246:** I developed the initial implementation of the Minimax algorithm and the core functionality enabling gameplay against the AI as well as AI vs. AI matches. I also contributed to the initial draft of the formal game representation and participated in reviewing and refining the report.
- s252362:** I implemented the Alpha-Beta pruning mechanism on top of the existing Minimax structure to improve search efficiency. I also contributed to the report writing, namely parts of the discussion chapter.
- s201974:** I led the development and formulation of the evaluation functions with help from the group. I contributed to the representation and state-space formulation and the discussion section. I also reviewed the code to align with report.

1.2 Percentage Contribution

Table of Contributions:

Emma	Thorri	Iben	Noa
25%	25%	25%	25%

2 NOMENCLATURE

Marbles The game pieces that are moved on the board on a players turn

Pit All the slots in the board are pits

Kalahas The two score/store/goal pits

Ambos The six small pits that are not the kalahas

Kalaha score The amount of marbles in the kalaha

Sowing The act of distributing marbles counterclockwise from a chosen slot

3 PROGRAMMING KALAH

3.1 Game rules

We have chosen to follow the standard rules of Kalah (see appendix section 4), with the special rules: (1) Extra turn: If the last marble lands in the player's Kalah, the player moves again. (2) Capture: If the last marble lands in an empty pit on the player's side, opposite marbles are captured. (3) Termination: The game ends when one side is empty, and remaining marbles are collected to the current player's kalah.

3.2 Formal Game Characterization

Properties Kalah is a turn-based two-player, deterministic, competitive, zero-sum game. It has perfect information as both players have full observability of the board state at all times. Kalah is zero-sum game, as any advantage gained by one player (represented by marbles in their kalah) is a direct loss for the opponent.

It can be modeled as a discrete state-space tree, with characteristics well suited for Minimax with Alpha-Beta pruning. However, the tree does not exhibit strictly alternating min and max layers (see section 3.4).

State Space Estimation The game consists of 12 slots and 2 kalahs (14 total slots) containing a total of 72 marbles ($6 \cdot 12 = 72$). A game state is defined by the distribution of these indistinguishable marbles across the 14 slots and the information of which player has the next turn.

An upper bound on the number of such distributions is obtained using the formula for combinations with repetition, which counts the number of ways to distribute n identical objects into k distinct categories having p different players, For $n = 72$ marbles and $k = 14$ slots and $p = 2$ players:

$$p \cdot \binom{n+k-1}{k-1} = p \cdot \frac{(n+k-1)!}{n!(k-1)!} = 2 \cdot \binom{72+14-1}{14-1} = 2 \cdot \binom{85}{13} \approx 1.48 \cdot 10^{15} \quad (1)$$

Thus the state space size is roughly on the order of 10^{15} . This is an upper bound since many states are not reachable due to the game's move constraints and the end-of-game-rules. Nevertheless, it illustrates that the state space is extremely large. While the state space is extremely large, the branching factor is relatively small, with at most 6 actions to choose from for the current player.

The size of the state space is too big to search to terminal nodes from the start, we therefore use a depth-limited Minimax search combined with a heuristic evaluation function to estimate the value of non-terminal leaves.

3.3 Implementation and State Representation

We define the game state as a tuple

$$s = (\mathbf{B}, p),$$

where $\mathbf{B}$ is a state vector of length 14 representing the board, and $p \in \{1, 2\}$ denotes the player whose turn it is to move.

The board indices are mapped as follows:

$x_{0,\dots,5}$: Slots belonging to Player 1	x_6 : Player 1's Kalaha(k_1)
$x_{7,\dots,12}$: Slots belonging to Player 2	x_{13} : Player 2's Kalaha(k_2)

This representation is both compact and efficient, as it allows constant-time access to all pits and enables straightforward simulation of moves. Furthermore, since Kalaha is a perfect-information game, it is sufficient to represent only the physical board configuration and the current player.

The initial state is defined as:

$$s_0 = ([6, 6, 6, 6, 6, 6, 0, 6, 6, 6, 6, 6, 0], p^{init}), \quad \text{where } p^{init} \text{ is randomly chosen from } \{1, 2\}$$

$PLAYER(s)$ returns the player (MIN or MAX) whose turn it is in state s . Corresponding to the player aiming to minimize or maximize the evaluation function in the minimax algorithm.

$ACTIONS(s)$ Given a state s it returns the indices of the slots on the current player's side that contain at least one marble.

$$ACTIONS(s) = \{i \mid x_i > 0\}, \text{ where } i \in \begin{cases} 0, \dots, 5 & \text{if } PLAYER(s) = MIN \\ 7, \dots, 12 & \text{if } PLAYER(s) = MAX \end{cases}$$

$RESULT(s, a)$ The transition function $RESULT(s, a)$ returns the successor state s' by simulating the "sowing" process:

1. Remove all stones from slot a .
2. Distribute them counterclockwise into subsequent slots and the player's own kalaha while skipping the opponent's kalaha.
3. Apply special rules:
 - If the last marble lands in the current player's Kalaha (x_6 or x_{13}), the turn indicator p remains unchanged.
 - Captures: If the last marble lands in an empty slot on the player's own side, the marble and those in the opposite slot are moved to the player's Kalaha.
 - If all slots on one side become empty, the remaining marbles on the opponent's side are moved to that player's kalaha and the game ends.

The result function is deterministic, meaning that each action produces exactly one successor state.

Terminal Test and Utility The game reaches a terminal state $TERMINAL(s)$ when all slots on one side are empty. At this point, any remaining marbles on the opposite side are added to that player's Kalaha.

$$TERMINAL(s) = true \iff \left(\sum_{i=0}^5 x_i = 0 \right) \vee \left(\sum_{i=7}^{12} x_i = 0 \right) \quad (2)$$

Since Kalaha is a deterministic game with perfect information, the full state of the game is always observable. Therefore, it is sufficient to represent physical states directly, and belief states are not

required.

The utility is determined by the final marble count in the Kalahas. To facilitate the Minimax search, we define a zero-sum utility function from the perspective of the MAX player, and defined as the difference between the amount of marbles in players' kalahas:

$$UTILITY(s) = k_{MAX} - k_{MIN} = x_{MAXkalaha} - x_{MINkalaha}$$

Thus each player attempts to maximize their kalahas score relative to the opponent.

Data Structures and Mapping The game board is implemented as a 14-element integer array. This structure allows for an efficient implementation of the "sowing" process: the next slot index is always $(i + 1) \pmod{14}$. To map player actions to this vector, we use a relative-to-absolute indexing system. While both players choose an action $a \in \{0, \dots, 5\}$, the engine translates this to index $i = a$ for Player 1 and $i = a + 7$ for Player 2. This ensures that the core game logic remains identical for both agents.

3.4 AI Algorithm: Minimax with Alpha-Beta Pruning

Algorithm Choice The choice of Minimax with Alpha-Beta pruning is appropriate due to the deterministic, zero-sum, and perfect-information nature of Kalaha. These properties ensure that optimal play can, in principle, be derived through adversarial search. Given the state space of 10^{15} , we cannot traverse to terminal nodes. Instead, we perform a depth-limited search to depth d , utilizing a heuristic evaluation function to approximate utility at the leaves. Alpha-Beta pruning is employed to enhance efficiency. The Minimax algorithm for optimal decision-making in adversarial games is found and described in the course book by Russell and Norvig (Chapter 5).

Because we have implemented the *extra-turn* rule, the same player may take multiple consecutive turns. This breaks the usual alternation between MAX and MIN in the minimax tree. Instead, our algorithm determines the active player at each node using the `current_player` attribute of the `GameState`. If `Result(s, a)` returns a state where the same player moves again, the algorithm continues maximizing or minimizing accordingly. Despite this, Alpha-beta pruning remains valid, as it relies only on the maximizing and minimizing objectives at each node, not on strict alternation between players.

Search Tree Example The search tree begins at the current board state s . Nodes represent board configurations, and edges represent legal moves from $ACTIONS(s)$. At MAX nodes, we compute $\max(v, \text{child_value})$. At MIN nodes, we compute $\min(v, \text{child_value})$. Pruning occurs when $\beta \leq \alpha$, allowing the algorithm to skip calculations on sub-trees that cannot improve the current minimax value. We have drawn an instance of the MINIMAX algorithm with alpha-beta-pruning in Appendix (section 5).

Heuristic/Evaluation Function This defines a mathematical function that estimates the "goodness" of a non-terminal state. The evaluation function $Eval(s)$ estimates the desirability of a state s for the AI. We have defined 4 different heuristics (evaluation functions). The most important component of the evaluation is the difference between the two kalahas, which is therefore part of all 4 functions.

The simplest and first evaluation function only evaluates the difference in the two kalahas, as this

is the main goal of the game:

$$Eval_1(s) = x_{MAXkalaha} - x_{MINkalaha} \quad (3)$$

This heuristic indirectly favors moves where "capture rule" is activated, when this resulting gain is visible in the search depth.

In addition to the difference in the kalahas, we have made a heuristic that additionally considers the difference in marbles remaining on each side of the board:

$$Eval_2(s) = w_1 \cdot Eval_1(s) + w_2 \left(\sum_{i \in MIN \text{ side}} x_i - \sum_{j \in MAX \text{ side}} x_j \right) \quad (4)$$

where w_1 and w_2 are positive weights that determine the relative importance of the features.

The second term in this heuristic favors states where the player has fewer marbles on their side. This indirectly encourages positions that can lead to collecting the opponent's remaining marbles, the "empty board rule" a late game strategy. Therefore, we have chosen to make w_2 dynamic. It increases as the number of marbles that are not in kalahas decrease. Hereby we define:

$$w_2 = 1 - \frac{M - (x_{MAX \text{ kalaha}} + x_{MIN \text{ kalaha}})}{M} \in [0, 1], \quad \text{where } M = \text{total number of marbles in the game.}$$

We set $w_1 = 1$ as we still want the first term to have more or equal weight than the second.

The third heuristic considers the number of possible "extra turn rule" activations of the state, in addition to the difference in the kalahas:

$$Eval_3(s) = w_1 Eval_1(s) + w_3 \cdot (T(s, p_{MAX}) - T(s, p_{MIN})) \quad (5)$$

where:

$$T(s, p) = \begin{cases} |\{i \in MIN \text{ side} \mid \text{where action } i \text{ ends in } MIN \text{ kalaha}\}| & \text{p is the MIN player} \\ |\{i \in MAX \text{ side} \mid \text{where action } i \text{ ends in } MAX \text{ kalaha}\}| & \text{p is the MAX player} \end{cases} \quad (6)$$

The second term in the heuristic (with weight w_3) counts moves that result in additional turn(s) of the MAX player, thereby favoring positions that allow the player of the state to retain control of the game.

$w_3 = \lambda \cdot \frac{M - (x_{MAX \text{ kalaha}} + x_{MIN \text{ kalaha}})}{M} \in [0, 1]$, where M is the total number of marbles in the game. This term lies in the interval $[0, \lambda]$ and decreases as more marbles are collected in the kalahas.

This reflects that extra-turn opportunities are more important in the early stages of the game, where there is greater flexibility to exploit them, while their impact diminishes in the endgame where fewer moves remain.

The scaling factor λ controls the overall influence of the feature. Based on preliminary experiments, we set $\lambda = 3$, which provided a reasonable balance between emphasizing extra-turn opportunities and maintaining stability in the evaluation function.

Lastly we introduce a fourth evaluations function. As before we include the difference in the kalahas, but also a term focusing on the "capture rule".

$$Eval_4(s) = w_1 \cdot Eval_1(s) + w_4 \cdot K, \quad \text{where } K = \sum_{i \in \text{MAX side}} \mathbf{1}\{x_i = 13\} - \sum_{i \in \text{MIN side}} \mathbf{1}\{x_i = 13\} \quad (7)$$

Using the same logic as for w_3 , we define: $w_4 = \beta \cdot \frac{M - (x_{\text{MAX kalaha}} + x_{\text{MIN kalaha}})}{M} \in [0, 1]$, and after experimenting we chose $\beta = 8$. Again focusing on early game, where the reward can be high with pits having 13 marbles, which activates the "capture rule".

3.5 Performance results and Benchmarking

To ensure fairness in the experiments, the starting player is randomized for each game.

Depth	Algorithm	Win rate	Avg. score diff.	Avg. time (s)	Nodes expanded	Cutoffs
2	w/o pruning	48%	0.16	0.0003	680	0
	w. pruning	52%		0.0002	462	69
4	w/o pruning	56%	0.60	0.0060	10295	0
	w. pruning	44%		0.0020	3112	657
6	w/o pruning	54%	0.96	0.2400	376836	0
	w. pruning	46%		0.0310	46369	11104

Table 2: Table of performance comparison between Minimax with and without Alpha-Beta pruning over 50 games using $Eval_1(s)$ at search depths of 2, 4, and 6. The table also includes performance metrics.

Exp	P2 Eval	P1 Wins	P2 Wins	Draws	P1 Win %	Score Diff	Time (P1/P2)
2	$Eval_2$	30	17	3	60%	10.56	0.088 / 0.086
3	$Eval_3$	19	21	10	38%	0.84	0.094 / 0.091
4	$Eval_4$	26	23	1	52%	3.72	0.094 / 0.096

Table 3: Table compares the performance of more complex evaluation functions against the $Eval_1$ baseline using a time-limited (0.1s) iterative deepening search.

3.6 Discussion and Future Work

As shown in Table 2, increasing the search depth d incurs a significant computational cost, with the number of expanded nodes growing exponentially (from 680 to 376.836 for plain minimax). Alpha-beta pruning mitigates this by substantially reducing both the number of explored nodes and computation time, enabling deeper search within practical limits. To balance depth and efficiency, we impose a time limit of $T_{\text{lim}} = 0.1$ s per move combined with iterative deepening. In Table 2, standard Minimax appears to outperform Alpha-Beta pruning at depths 4 and 6 (e.g., a 56% vs. 44% win rate at depth 4). While this difference is primarily due to move ordering — where the algorithms may explore different branches at a fixed depth — it is important to note that these "Win Rate" differences are not definitively representative. Given the relatively small sample size of 50 games, these fluctuations are likely not statistically significant and fall within the expected margin of error.

It should be noted that Kalaha exhibits a starting-player bias, meaning that some of the observed performance differences may be influenced by which player moves first.

The results in Table 3 demonstrate that increasing heuristic complexity does not inherently improve performance. *Eval₃* was the only variant to outperform the *Eval₁* baseline, achieving a 52% win rate (calculated from a 38% win rate for P1 across 50 games, with 10 draws) and the only positive average score difference (+0.84). In contrast, *Eval₂* and *Eval₄* both resulted in negative score differences and failed to beat the simpler baseline. This suggests that while capturing structural patterns — such as extra-turn opportunities accounted for in *Eval₃* — can provide a marginal advantage, many additional features introduce "noise" that fail to outweigh the benefits of a simple Kalaha difference. Furthermore, the potential benefits of complex functions may not be fully realized without effective move ordering to improve pruning efficiency. Given the sample size of 50 games and the inherent starting-player bias, these marginal differences may not be statistically significant and should be interpreted with caution.

Overall, the results highlight a trade-off between evaluation function complexity and search depth, where deeper search tends to have a greater impact on performance than more elaborate heuristics.

Non-alternating Minimax algorithm Because the "extra-turn" rule allows a single player to move multiple times in a row, a fixed search depth d does not consistently correspond to a fixed number of game rounds. For instance, at a search depth of $d = 4$, a sequence of three extra turns by the MAX player would result in the algorithm looking only one "round" ahead (one move for MIN). We handled this by letting the AI continuously increment the search depth ($d = 1, 2, 3, \dots$) until a timelimit on 0.1 s was reached. At this limit, the algorithm breaks the loop and returns the best action found from the previously completed depth. This effectively bypasses the depth imbalance caused by extra turns, ensuring the engine always utilizes maximum allowed computation time to search as deep as possible.

Future work: implementing Monte Carlo Tree Search (MCTS) MCTS differs from Minimax as there is no heuristic required. In MCTS, the "value" of a state is not an estimate, but the statistical average of many playouts. By performing many of these playouts, the AI develops a statistical model of which move in Kalaha is most likely to result in a victory, rather than just which move leads to the highest immediate marble count. Furthermore, the MCTS would not stop at a fixed depth. MCTS can "look" much deeper into the gameplay if a certain path is very promising, effectively bypassing the "horizon effect" as mentioned in slides week 6. Compared to Minimax, MCTS requires thousands of playouts to accurately evaluate a single state. This can be computationally expensive and may require significantly more time compared to a depth/time-limited Minimax search as we have implemented.

3.7 Conclusion

We implemented Kalaha and developed an AI using Minimax with Alpha-Beta pruning. Due to the large state space, depth-limited search with heuristic evaluation was necessary. The results show that search depth has a stronger impact on performance than heuristic complexity, while Alpha-Beta pruning significantly improves efficiency. Overall, the project demonstrates how adversarial search methods can effectively be applied to board games under practical computational constraints.

4 APPENDIX A

Kalaha is a board game for two players, played on a board with 14 slots. Each player has six small slots (called ambos) on their side and one larger slot called the kalaha (goal slot).

Setup At the beginning of the game, 6 marbles are placed in each of the 12 small slots. The two kalahas are empty.

Gameplay The goal of the game is to get more marbles in your kalaha than your opponent.

On your turn

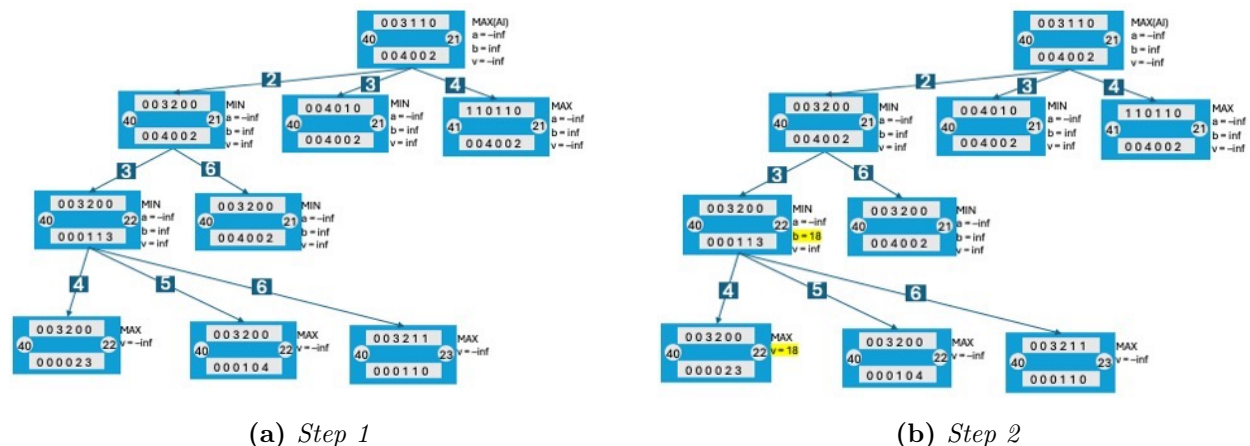
- Choose one of the six small slots on your side. Pick up all the marbles from that slot.
- Moving counterclockwise around the board, you drop one marble into each subsequent slot.
- You place marbles into your own small slots, your own kalaha, and your opponent's small slots.
- You skip your opponent's kalaha and do not place a marble there.

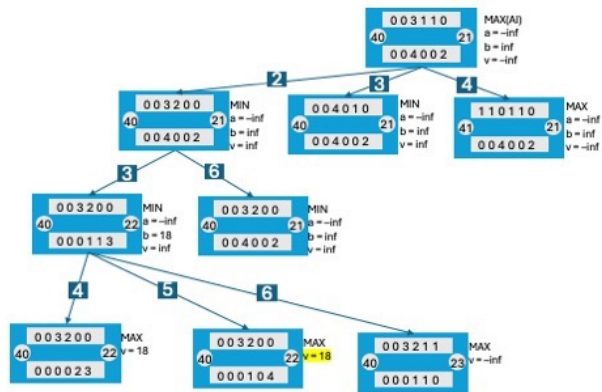
End of the Game The game ends when all marbles are in either of the two kalahas.

Winning the Game The player with the most marbles in their kalaha wins. If both players have the same number of marbles, the game is a draw.

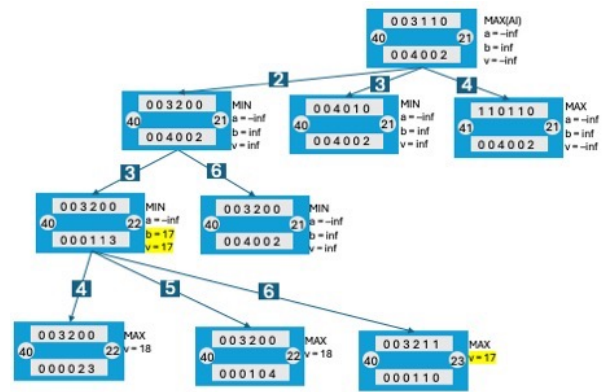
5 APPENDIX B

Figure I: MINIMAX algorithm with α - β pruning. Using $d = 4$ and $v = \text{Eval}(s) = x_{13} - x_6$. In each step the changes are highlighted in yellow, and a pruning indicated with a red arrow. The algorithm is run until reaching the first pruning.

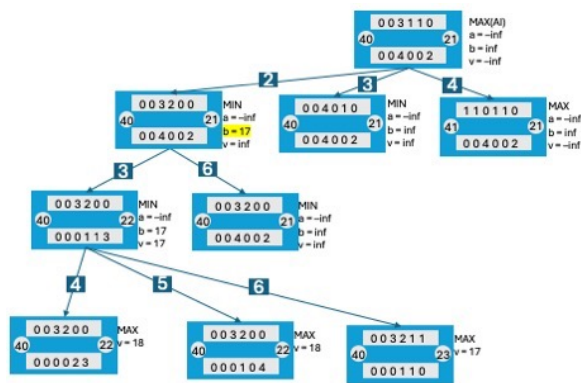




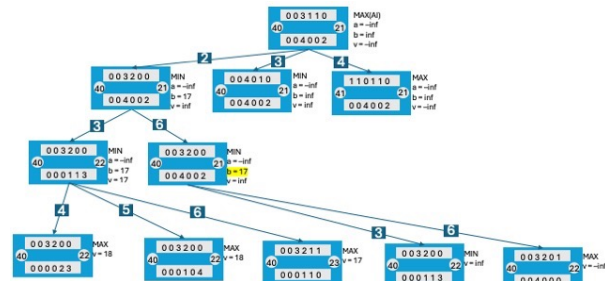
(c) Step 3



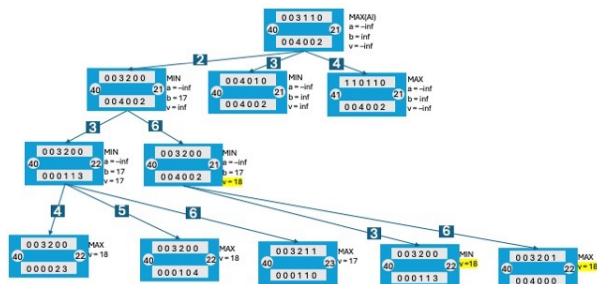
(d) Step 4



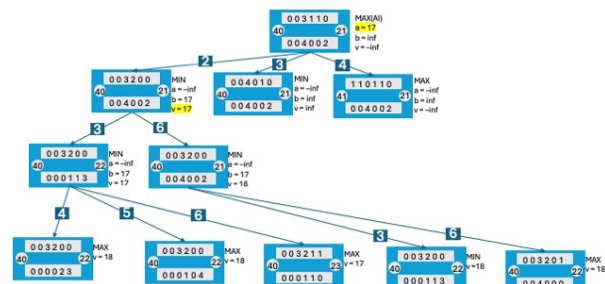
(e) Step 5



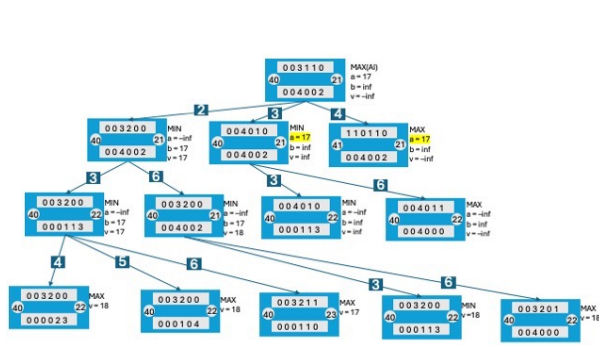
(f) Step 6



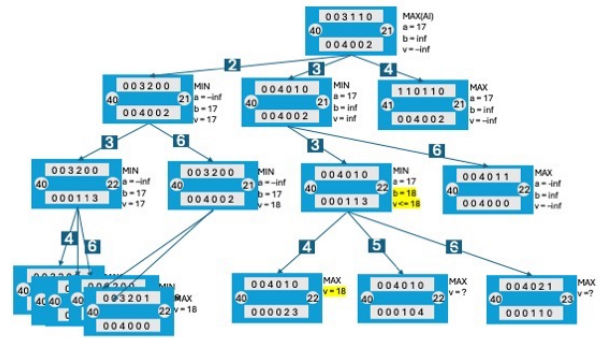
(g) Step 7



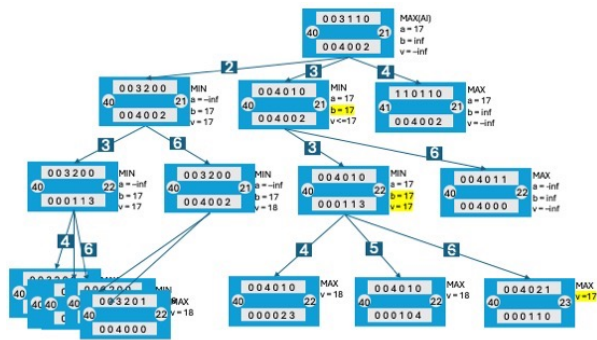
(h) Step 8



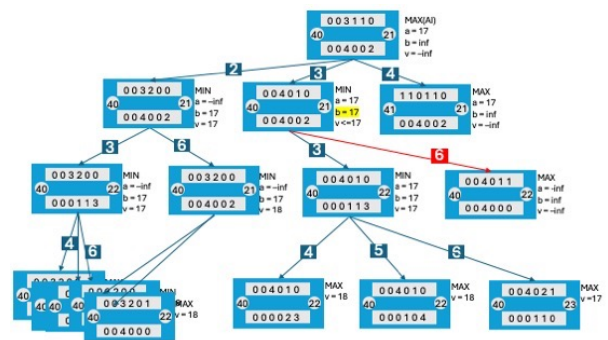
(i) Step 9



(j) Step 10



(k) Step 11



(l) Step 12